SCSE2133
Additional Notes
User Stories

Rohayanti Hassan
Faculty of Computing
Universiti Teknologi Malaysia

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

User Stories

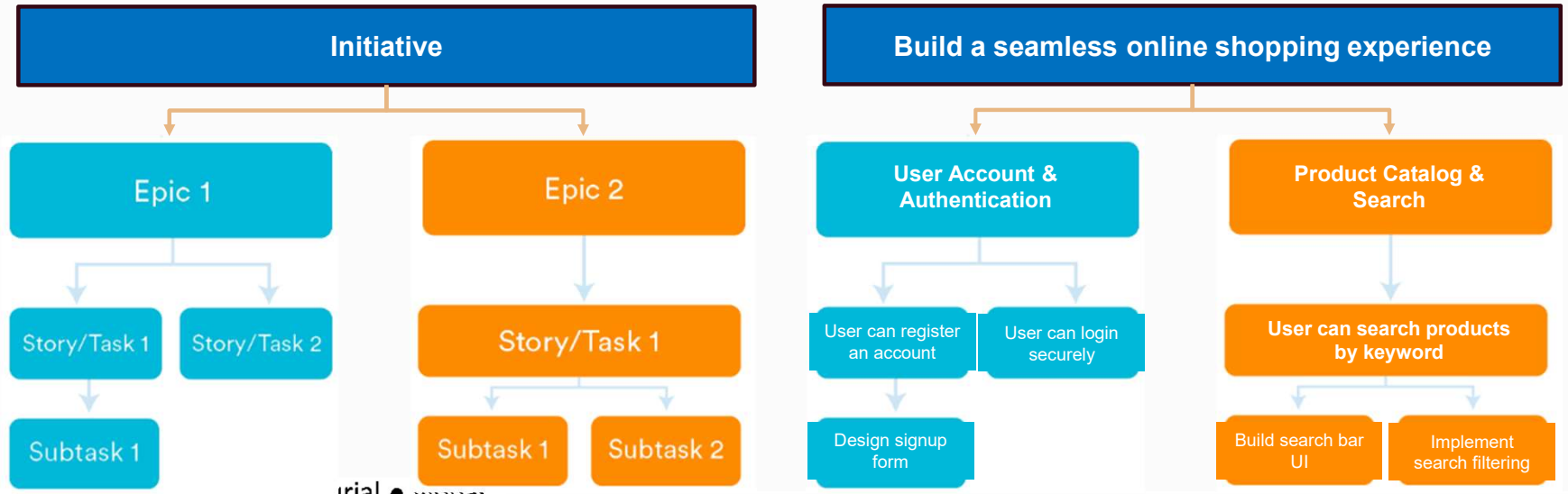
Introduction: User Story and Agile

- A user story is an informal, general explanation of a software feature written from the perspective of the end user. Its purpose is to articulate how a software feature will provide value to the customer.
- User stories are one of the core components of an agile program. it drives collaboration, clarity, and better products.
- Written in non-technical language, it keeps the end user at the center. It explains what is built, why, and the value for the customer.

Working with User Story

- **Scrum**: User stories go into sprints, driving estimation, planning, and forecasting.
- **Kanban**: User stories move through workflow, helping manage WIP and refine processes. In both, stories improve agility and efficiency.

User stories in a building block
Initiative → Epic → Story



Variations of User Story Format

The common format ensures every user story explains the who, what, and why.

Format	Structure	Example	Best Use Case
Role–Feature–Reason	*As a [role], I want [feature] so that [reason/benefit].*	*As a shopper, I want to save items in a wishlist so that I can purchase them later.*	Simple, user-focused; best for **capturing user needs quickly**.
Collaboration-based Approach	*As a [intended user]* *I want [intended action],* *so that [purpose/result of the action, profit achieved],* *Acceptance Criteria:*	*As a **customer of the bank*** *I want to **log into the system*** *So that **I can use bank products*** ***Acceptance criteria:*** • *Login must be a valid email address.* • *System rejects login attempt with incorrect password.* • *System rejects login attempt for non-existent user login.*	Ensures **clear testable conditions**; best for defining **acceptance criteria**.
Testable Format (BDD)	*Scenario → Given [initial context], When [event], Then [outcome].*	*Scenario: Add product to cart → Given I am on a product page, When I click "Add to Cart", Then the item appears in my cart.*	Best for **detailed testing** and collaboration between **devs, testers, and business**.

Example of User Story
COLLABORATION-BASED APPROACH

User story US-002-02
As a logged-in bank customer
I want to be able to transfer money to another account
So that be able to transfer funds between accounts

Acceptance criteria
(AC1) the amount of the transfer cannot exceed the account balance
(AC2) target account must be correct
(AC3) for valid data (amount, account numbers) the source account balance decreases, and the destination account balance increases by the amount of the transfer
(AC4) the transfer amount must be positive and represent the correct amount of money (i.e., be accurate to two decimal places max)

🔷 How it might look in Jira

Epic: Banking Transactions
Story: US-002-02 Transfer Money Between Accounts
Task: Frontend – Transfer Form
- Sub-task: Design form
- Sub-task: Implement form
- Sub-task: Add client-side validation
- Sub-task: Display messages

Task: Backend – API & Business Rules
- Sub-task: Create transfer API
- Sub-task: Implement validation rules
- Sub-task: Update balances
- Sub-task: Log transactions

Task: Database Updates
- Sub-task: Schema changes (if any)
- Sub-task: Rollback mechanism

Task: Testing
- Sub-task: Unit tests
- Sub-task: Integration tests
- Sub-task: QA test scenarios

Writing Effective User Story with Acceptance Criteria

- Acceptance criteria can be viewed as test conditions or coverage items that should be checked by acceptance tests.

- Acceptance criteria are used:
  - To define the boundaries of the user's story
  - To reach a consensus between the developer team and the client
  - To describe both positive and negative test scenarios
  - As the basis for user story acceptance testing

- The two most common formats are:
  - Scenario-oriented acceptance criteria - Given/When/Then style criteria
  - Rule-oriented acceptance criteria

Writing Effective User Story with Acceptance Criteria

Rule-Oriented Example

Acceptance criteria:

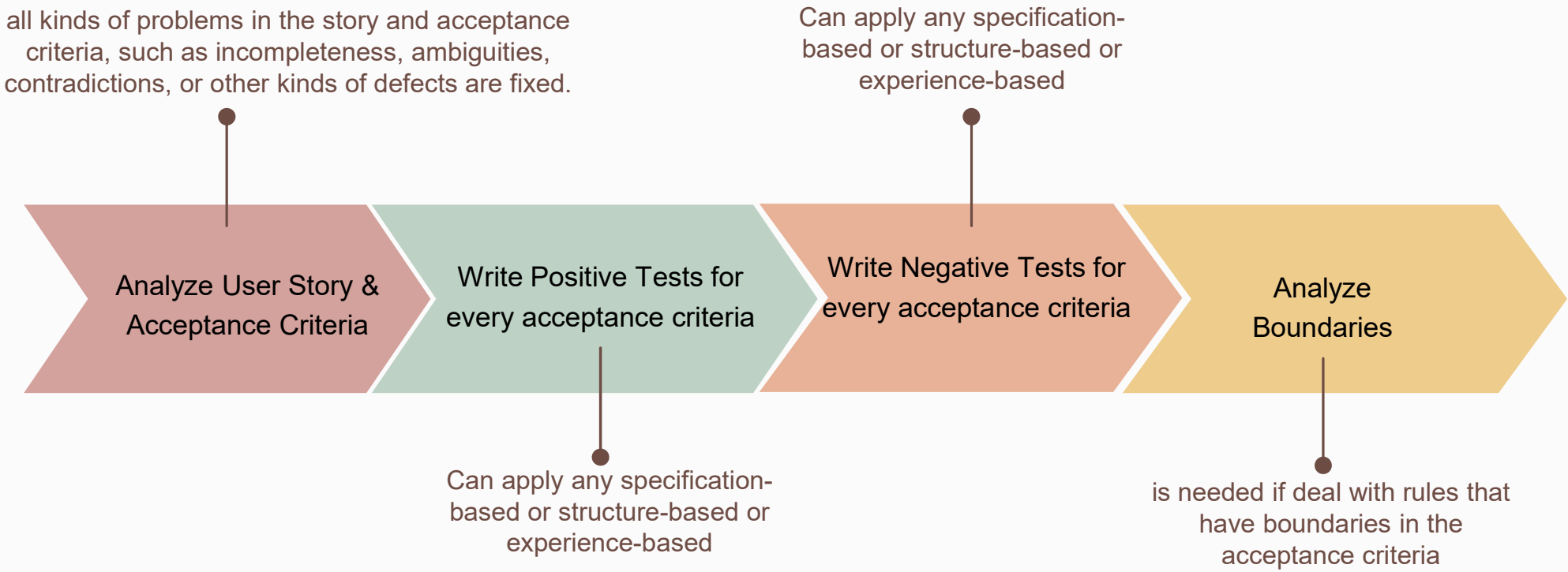
- Login must be a valid email address.
- System rejects login attempt with incorrect password.
- System rejects login attempt for non-existent user login.
- System allows the user to enter in the fields “login” and “password” only alphanumeric characters as well as a period and the symbol “@”.

Scenario-Oriented Example

Scenario:
There are two types of cards: one with a debit limit of $2500, the other with a debit limit of $5000. The maximum credit card limit depends on the customer’s earnings (rounded to the nearest $). The customer must have a salary of more than $10,000 per month to get the lower debit limit. If the salary exceeds $15,000 per month, the customer gets a higher debit limit.

Given a customer with monthly earnings <earnings amount>.
When a customer applies for a credit card.
Then the application should be <accepted> or <rejected> and if it is accepted, the maximum credit card limit should be <maximum limit>.

Writing the Acceptance Criteria



Example: Suppose a team intends to test the following user story

User story US-002-02
As a logged-in bank customer
I want to be able to transfer money to another account
So that be able to transfer funds between accounts

Acceptance criteria
(AC1) the amount of the transfer cannot exceed the account balance
(AC2) target account must be correct
(AC3) for valid data (amount, account numbers) the source account balance decreases and the destination account balance increases by the amount of the transfer
(AC4) the transfer amount must be positive and represent the correct amount of money (i.e., be accurate to two decimal places max)

Derive positive test condition for each acceptance criteria (AC)

Example: Positive test condition

COLLABORATION-BASED APPROACH

- Each AC or related ACs as a test condition for a “happy outcome”. Can apply the EP and BVA

- AC1+AC2+AC3+AC4
  - Transfer of the correct positive amount that does not exceed account balance to a correct targeted account for example 1500.00

- Can apply the EP or BVA
  - Transfer of the correct minimum possible (e.g., RM0.01) positive amount that does not exceed account balance to a correct targeted account
  - Transfer of the correct very large positive amount(e.g., RM8,438,483,784) that does not exceed account balance to a correct targeted account

Example: Negative test condition

COLLABORATION-BASED APPROACH

- For each AC, identify all possible negative cases that lead to an exception. Can apply the EP or BVA.
- (AC1) the amount of the transfer cannot exceed the account balance
  - transfer > balance to the correct targeted account.
  - When the transfer amount is greater by 1 cent (the minimum possible increment) than the account balance
- (AC2) The target account must be correct
  - transfer < balance to the non-existent targeted account
  - transfer < balance to the same account
- (AC4) the transfer amount must be positive and represent the correct amount of money (i.e., be accurate to two decimal places max)
  - The invalid decimal amount for example 3 decimal places
  - A negative number representing the correct amount (e.g., RM-150.25)
  - Number 0
  - A character string containing letters (e.g., RM15B.20)
  - An empty character string

Writing Effective User Stories: Tips

Effective user stories are concise but meaningful. They are written collaboratively, refined regularly, and backed with acceptance criteria to avoid ambiguity.

- Keep them short and simple
- Focus on value, not tasks
- Collaborate with users to refine
- Add acceptance criteria for clarity



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my